Universidade Federal de Minas Gerais
Instituto de Ciências Exatas
Departamento de Ciência da Computação
DCC642 – Introdução à Inteligência Artificial (2026/2)

Trabalho Prático 1 – Busca Competitiva

Data de entrega: 14/10/2026, 23:59h

1 Introdução

Neste trabalho, você irá implementar um agente de IA para jogar **Ligue-4** (*Connect Four*) utilizando algoritmos de **busca competitiva**. Para isso, você utilizará uma página web acessada por meio de um servidor web local, executado em seu próprio computador. A partir dessa página, será possível jogar humano vs. IA, observar partidas IA vs. IA e realizar testes controlando parâmetros como a profundidade máxima e o tempo máximo por jogada.

O objetivo principal é praticar a implementação do **Minimax**, da **poda Alfa-Beta** e do **Iterative Deepening**, em conjunto com uma **função heurística de avaliação**. Além de codificar o agente, você deverá redigir um **relatório** descrevendo sua metodologia e analisando os resultados experimentais obtidos.

2 Código-base

2.1 Executar o servidor

O código-base do trabalho está **anexado à tarefa no Moodle**. Ele inclui o cliente em JavaScript (p5.js) para visualizar e jogar Ligue-4, o servidor em Python (Flask) e o arquivo `search.py`, onde você implementará o agente. Para executar o servidor, instale as dependências e inicie-o com os seguintes comandos:

Terminal

```
python -m venv tp1-env          # Criar ambiente virtual Python
source tp1-env/bin/activate     # Ativar ambiente virtual
pip install -r requirements.txt
python server.py                # Executar o servidor (porta padrão: 5001)
```

Assim que o servidor iniciar, abra no navegador o endereço `http://localhost:5001`. Essa URL deve mostrar a interface do jogo Ligue-4, como na Figura 1.

2.2 Interface do jogo

A interface exibe o tabuleiro 6×7, indicadores de vez e de vencedor, além dos seguintes controles:

- **Selecionar jogadores:** o jogador 1 (P1) e o jogador 2 (P2) podem ser tanto IAs quanto humanos.
- **Tempo por jogada:** limite de tempo (em ms) que seu agente deve respeitar.
- **Novo jogo:** reinicia a partida.

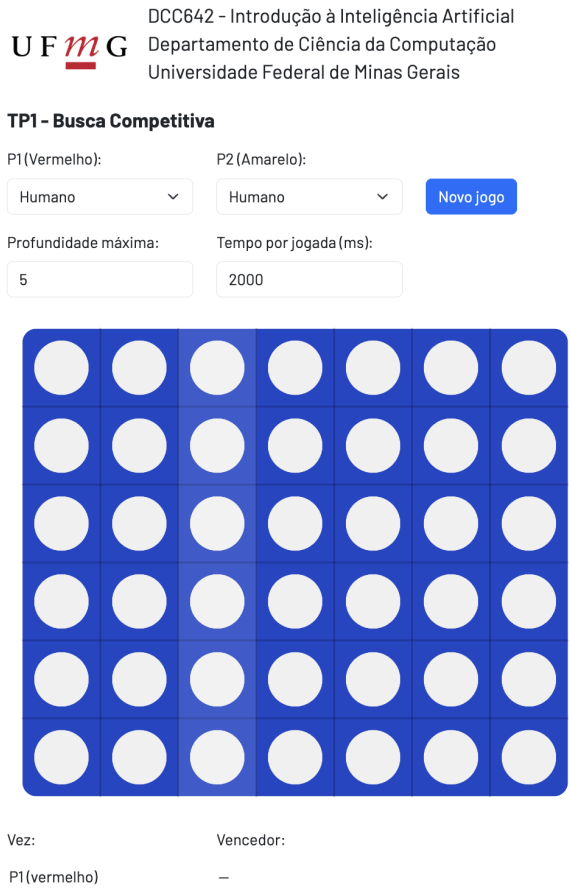


Figura 1: Interface do jogo Ligue-4.

2.3 Estados e ponto de entrada

Todo o seu trabalho será realizado no arquivo `search.py`. O servidor chamará a função:

search.py

```
choose_move(board, player, config) → (coluna, info)
```

onde:

- `board` é uma matriz 6×7 com valores em $\{0, 1, 2\}$ (0 = vazio, 1 = Jogador 1, 2 = Jogador 2);
- `player` é o jogador da vez (1 ou 2);
- `config` contém o tempo máximo da jogada em milissegundos (`max_time_ms`) e a profundidade máxima da busca (`max_depth`);
- `coluna` é a coluna escolhida para a jogada e `info` reúne informações auxiliares sobre a busca (por exemplo, número de nós expandidos e profundidade atingida).

O código inicial do `search.py` escolhe uma jogada aleatória válida, apenas para validação do ambiente. Você deverá substituir essa lógica por uma busca competitiva com heurística.

3 Tarefas

3.1 Baseline (agente aleatório)

Valide o funcionamento do ambiente executando partidas Humano vs. IA (aleatória) e IA aleatória vs. IA aleatória. Essa etapa serve apenas para confirmar a comunicação cliente-servidor e a atualização do estado do jogo.

3.2 Minimax com profundidade limitada e heurística

Implemente o algoritmo Minimax com profundidade limitada. Para estados terminais, atribua valores extremos (vitória/derrota) e 0 para empate. Para estados não terminais (quando a profundidade limite é atingida), utilize uma função heurística que estime a "vantagem" do jogador da vez. Sugestões de componentes heurísticos:

- valorização do centro do tabuleiro;
- contagem de sequências (duplas e triplas abertas) e de ameaças;
- penalidades para oportunidades do oponente.

Experimente diferentes profundidades (por exemplo, 2, 3, 4 e 5) e observe o impacto no tempo e na qualidade da decisão.

3.3 Minimax com poda Alfa-Beta

Acrescente a poda Alfa-Beta à sua implementação. A poda deve reduzir o número de nós avaliados **sem alterar a decisão ótima** do Minimax. Registre o número de nós expandidos para comparação com a versão sem poda.

3.4 Iterative Deepening com limite de tempo

Implemente o Iterative Deepening: execute sucessivamente as profundidades 1, 2, 3, ... até atingir `max_depth` ou estourar `max_time_ms`. Mantenha sempre a melhor jogada conhecida. Recomenda-se combinar o Iterative Deepening com a poda Alfa-Beta e, se possível, com ordenação de jogadas (por exemplo, tentar a coluna central primeiro), para aumentar a profundidade atingida no mesmo tempo.

3.5 Competição

Depois de implementar a busca com Iterative Deepening, pense em técnicas para melhorá-la (tabelas de transposição, funções de avaliação melhores, ordenação dos movimentos etc.), pois haverá uma **competição** entre as IAs dos alunos! Os 3 primeiros lugares receberão **pontos extras**. As regras da competição são as seguintes:

- **Tempo por jogada:** 3 s (definido pelo professor). O servidor aplica um limite rígido de tempo.
- **Rodadas duplas:** dois jogos por confronto (invertendo quem começa). Pontuação: vitória = 1, empate = 0,5, derrota = 0.
- **Desclassificação técnica:** agentes que travarem ou violarem o tempo repetidamente podem receber uma jogada aleatória de *fallback* ou WO daquela jogada.

4 Relatório

Após concluir a implementação dos algoritmos de busca, você deverá executar experimentos e reportar os resultados encontrados. Os experimentos são os seguintes:

1. **Minimax vs. Aleatório**

- Profundidades: 2, 3, 4 e 5.
- Métricas: taxa de vitória, tempo médio por jogada e média de estados visitados.

2. **Alfa-Beta vs. Minimax (sem poda)**
   - Profundidades: 2, 3, 4 e 5.
   - Métricas: taxa de vitória, tempo médio por jogada e média de estados visitados.
3. **Iterative Deepening vs. Alfa-Beta**
   - Limites de tempo: 1 s e 2 s.
   - Métricas: taxa de vitória, tempo médio por jogada, profundidade média atingida e média de estados visitados.
4. **IA do aluno vs. jogador humano**
   - Jogue pelo menos 5 partidas contra a sua melhor IA.
   - Descreva percepções qualitativas (forças e fraquezas observadas).

O seu relatório deve conter **no máximo 3 páginas** no formato de duas colunas da *Association for the Advancement of Artificial Intelligence* (AAAI). O relatório deve conter:

1. **Introdução e Objetivo.**
2. **Metodologia:** evolução do agente (Minimax → Alfa-Beta → Iterative Deepening), heurística e decisões de projeto.
3. **Experimentos e Resultados:** tabelas e (opcionalmente) gráficos com as métricas propostas.
4. **Discussão:** análise crítica dos resultados, *trade-offs* e limitações.
5. **Conclusão:** síntese do que funcionou melhor e ideias de melhorias.

O template da AAAI-27 pode ser baixado em: `https://aaai.org/authorkit27`.

5 Submissão

Para submeter o seu trabalho, envie na tarefa "TP1: Busca Competitiva" no Moodle os seguintes arquivos separadamente, **sem compactá-los em formato .zip ou similar**:

1. `search.py` – o código-fonte do seu agente.
2. `relatorio.pdf` – o relatório, em formato PDF, com no máximo 3 páginas no formato AAAI. **Não é permitido alterar o formato!**

6 Uso de IA

Ferramentas de IA podem ser utilizadas como apoio na implementação (por exemplo, geração de código auxiliar, como funções de teste e de coleta de métricas) e na escrita do relatório, desde que o uso seja declarado no final do relatório, informando o que foi usado e com qual objetivo, com links para as conversas feitas.

Não é permitido o uso de IA para gerar automaticamente, na íntegra, a implementação dos algoritmos de busca (Minimax, Alfa-Beta e Iterative Deepening) e da função heurística, nem o relatório.